

Build night 5

Demo

Rehearsal

Three minutes, four parts, a plan for when the live call fails, and the evidence a judge opens.

Travis Sim

NUS · Singapore

Thu 11 Feb 2027 · COM3/COM4

Where we are

One week to Demo Day. Tonight we make the pitch reviewable.

DONE

Workshop 4

Thu 4 Feb

Performance scorecard and shipping

TONIGHT

Build night 5

Thu 11 Feb

Demo rehearsal and freeze

NEXT

Demo Day

Thu 18 Feb

Working projects, agent, exchange or tooling

Tonight

18:00 to 20:00. Two timed runs, one fix block, one freeze.

18:00 — 15 min
Readiness circle
presenters, links, missing evidence

18:15 — 20 min
Demo structure
four parts and the failure plan

18:35 — 30 min
Rehearsal 1
3 minutes plus 2 of questions each

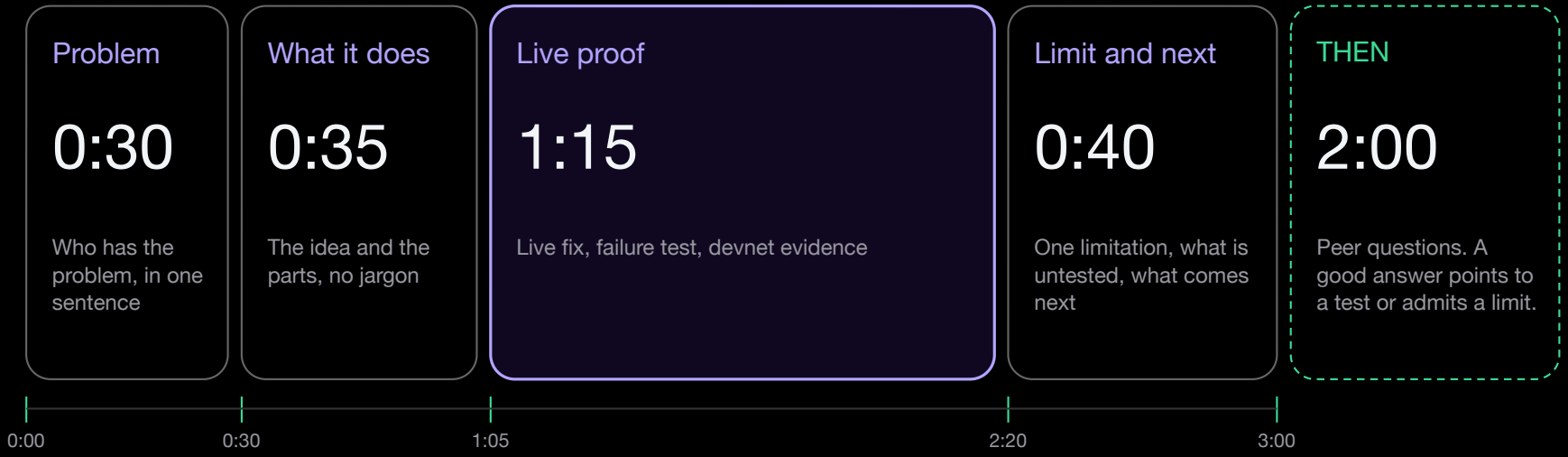
19:05 — 30 min
Fix the weakest claim
rerun the core test

19:35 — 20 min
Rehearsal 2
new listeners, recorded backup

19:55 — 5 min
Freeze and send
commit, demo pack to the leads

Three minutes, four parts

A budget for each part, then two minutes of questions.



Run over by more than 10 seconds in any part and the check list fails. Rushed under half and you are skipping something.

Source: demo-repo/code/out/pitch.json (budgets from code/src/timer.ts).

Part 1 and 2: problem, then what it does

About a minute together. No jargon, one sentence each.

PART 1: PROBLEM

“An agent that can place orders will sooner or later send a bad one: too big, based on a stale price, or in the wrong unit. Anyone building an agent on Solana needs a plain check between the idea and the action. Without it, one wrong number moves real money.”

50 words, about 23 s of 30 s

PART 2: WHAT IT DOES

“This is a small agent. It reads recorded Orca SOL/USDC swaps, cuts them into 20,000 dollar bars, wakes on an 8 basis point change alert and proposes one order per alert. A risk hook checks each order against the Pyth oracle and answers yes or no, with the rule that decided. Accepted orders go into a ledger.”

57 words, about 26 s of 35 s

Read from the sample script for the risk-hooked agent. Yours will differ; the shape should not.

Source: demo-repo/code/out/pitch.json (sample script, written from out/walkthrough.json).

Part 3: live proof

Seventy-five seconds. Show it working, show it refused, show where it lives.

LIVE FIX

One real decision

9 / 1

accepted / refused of 10 change alerts (1 max-notional)

Alert 8: sell 1,064 USD

`[max-notional] limit 1,000 USD`

FAILURE TEST

Bad orders, refused

- 5,000 USD order
  `REJECT max-notional`
- oracle 300 s old
  `REJECT max-oracle-age`
- price in cents
  `REJECT unit-sanity`
- 100 bps off the oracle
  `REJECT max-oracle-gap`

Injected on purpose; all four refused.

DEVNET EVIDENCE

A link, or why none

Signed memo tx

devnet node: AccountNotFound (no SOL), signature accepted

Flipped one bit

devnet node: SignatureFailure

Funded tx link

none: faucet answered HTTP 429

Devnet slot 513,629,831, read 2026-10-21. Say so; do not hide it.

Source: demo-repo/code/out/walkthrough.json and recorded/devnet-reads.json.

Part 4: limit and next

Forty seconds. Say what you have not proved.

SAY OUT LOUD

- One limitation, in your own words.
- What is untested.
- The next step, one sentence.

DO NOT SAY

- A return, a profit or a win rate as the headline.
- A capability the tests do not cover.
- A transaction link you have not opened tonight.

SAMPLE, PART 4

“Limits: this is simulated on recorded data, about seventeen minutes of one afternoon. The order sizes and the fee are assumptions, and no order was sent anywhere. The ledger is not evidence of returns. Next: write one funded devnet memo for every accepted order, and run the same hook on the live feed.”

Judges score utility, tests and devnet evidence, never simulated returns.

What the judge sees

Three questions, four checks each. Twelve in all.

UTILITY 4 / 4

- ● Names a user and a problem in one sentence
- ● Runs from a clean checkout using only the README
- ● Shows one decision end to end: input, rule, result
- ● States what it does not do

TESTS 3 / 4

- ● npm test passes and the report is attached
- ● A test covers the failure case (a bad input is refused)
- ● Tests run offline on recorded data
- ○ The test report is from the frozen commit

DEVNET EVIDENCE 2 / 4

- ● Shows a devnet address, or says there is none and why
- ○ At least one devnet transaction link opens
- ○ What the slide claims matches what the link shows
- ● Reads labelled public mainnet; writes labelled devnet only

Not scored: any simulated return. The score is the evidence.

Source: demo-repo/code/src/rubric.ts; sample pack scored in out/rubric.json.

Filled mint dot: the sample pack passes the check. Sample pack: 9 of 12.

One set of evidence

The same claim must be true in the pitch, the repo and the running demo.

ONE CLAIM, THREE PLACES

10 change alerts: 9 accepted, 1 refused (max-notional)

PITCH

Says what works, what failed, what is untested.

WHERE THIS CLAIM LIVES

Part 3 says: 9 accepted, 1 refused.

REPO

A frozen commit, a README that runs, a test report.

WHERE THIS CLAIM LIVES

out/walkthrough.json: rejectedByRule max-notional: 1. A test checks it.

RUNNING DEMO

Live, or the recorded copy of the same output.

WHERE THIS CLAIM LIVES

Demo 4, alert 8: REJECT [max-notional].

Source: demo-repo/code/out/walkthrough.json.

The sample project

Built from earlier sessions, run on recorded data. Rehearse on it before your own.

REAL SWAPS

1,114

Orca SOL/USDC, recorded

→

DOLLAR BARS

51

$20,000 each

→

CHANGE ALERTS

10

8 bps threshold

→

ORDERS

9 / 1

accepted / refused by the risk hook

→

LEDGER

9

fills, next swap price

WINDOW

18:13:46 to 18:30:20 UTC, 21 Oct 2026

2,900 swaps recorded; 1,114 used after cleaning and the Pyth window

SCORECARD (NOT SCORED BY JUDGES)

+3.00 USD net, hit rate 67%, drawdown 8.03 USD

Simulated, on recorded data; not advice. Tests: 51 passing, 0 failing.

Reads use public mainnet data; every write is devnet only; no real funds.

Source: demo-repo/code/out/walkthrough.json; recorded data from recorded/SOURCES.md. Simulated; not advice.

The decision in code

demo-repo/code/src/agent.ts

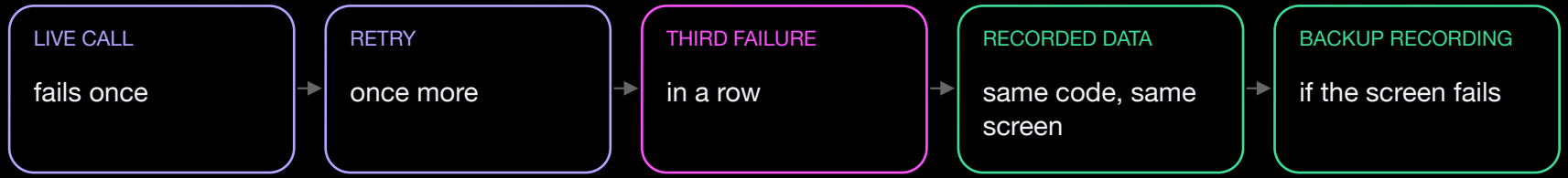
```
const side: Side = ev.direction === "up" ? "buy" : "sell";
const sizeUsd = Math.round(bucketBps * params.usdPerBps);
const order: Order = { side, sizeUsd, price: ev.price };
const oracleAgeSec = Math.max(0, ev.t - read.publishTime);
const context: RiskContext = { oraclePrice: read.price, oracleAgeSec };
const decision = checkOrder(order, context, params.limits);
if (!decision.ok) return;
```

1. The alert proposes an order.
2. The oracle gives the context.
3. The risk hook says yes or no, with the rule.

Source: demo-repo/code/src/agent.ts (excerpt: comments, blank lines and two bookkeeping lines removed)

When a live call fails

One retry, then recorded data, then the recording.



THE RULE

A failed call is tried again. The third failure in a row switches to recorded data.

Back on track within 30 seconds.

WHAT YOU SAY, THEN CARRY ON

“The live call is not answering, so I am switching to recorded data from 2026-10-21. It is the same logic on data I captured earlier.”

“Same input shape, same rule, same refusal. Here is the decision.”

Source: demo-repo/code/src/drill.ts and out/drill.json.

Demo: rehearsal timer

PREDICT FIRST

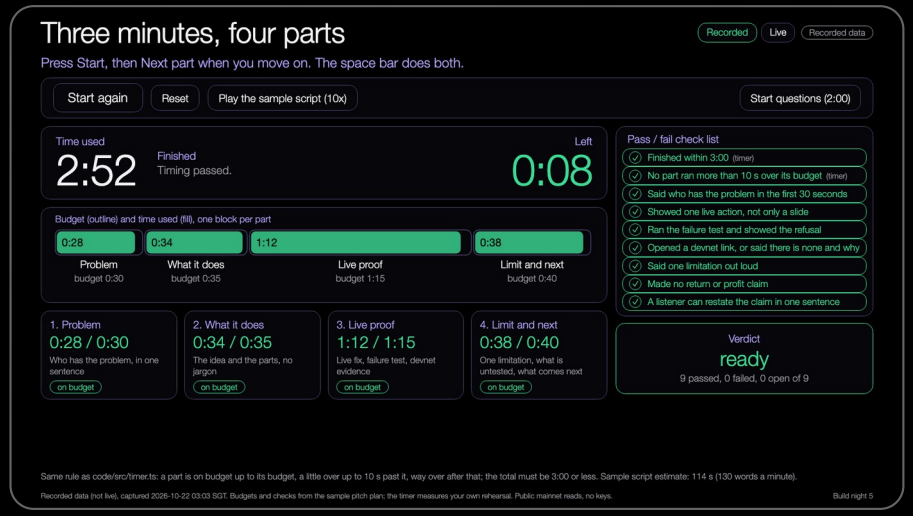
With the budgets on the slide, which part do you usually run over?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- four parts with a budget and a status each
- a verdict from the nine-item check list

Fallback: The timer needs no network. Use the space bar.



Lab demo, runs from disk, recorded data.

`Try it: https://nus-solana-demos.vercel.app/0009/01-rehearsal-timer.html`

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data)

Demo: what the judge sees

PREDICT FIRST

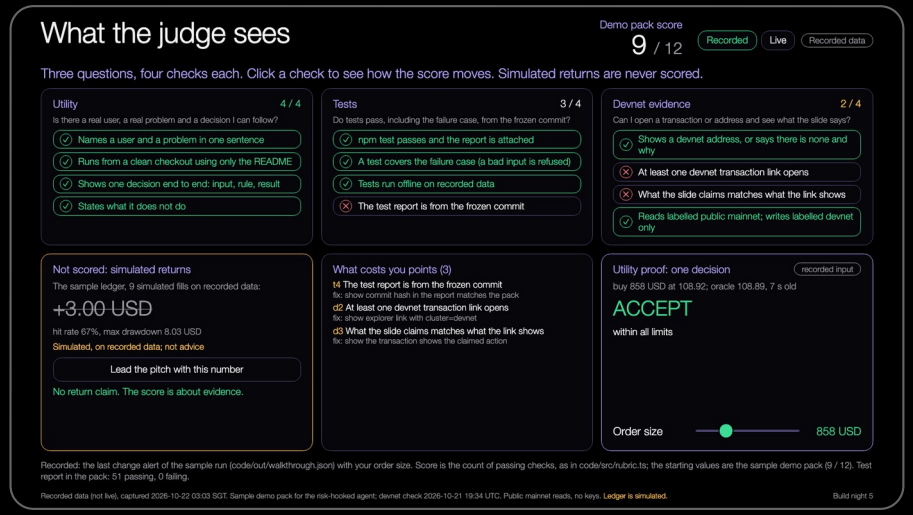
Which two checks does the sample pack miss, and why?

```
$ npm run report && npm run demo -- --recorded
```

YOU SHOULD SEE

- 12 checks, a score and a fix list
- a simulated return that never changes the score

Fallback: Recorded data is the default. The rubric needs no network.



Lab demo, runs from disk, recorded data.

Try it: https://nus-solana-demos.vercel.app/0009/02-judge-view.html

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data)

Demo: live to recorded switch

PREDICT FIRST

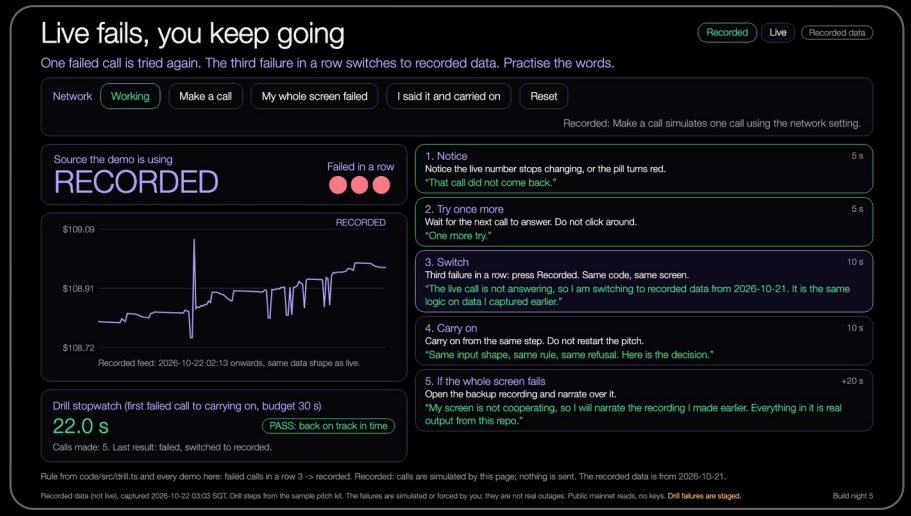
How many seconds from the first failed call to carrying on?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- three failed calls in a row, then the switch
- a stopwatch against the 30 second budget

Fallback: Recorded mode simulates the failures. Nothing is sent.



Lab demo, runs from disk, recorded data.

`Try it: https://nus-solana-demos.vercel.app/0009/03-switch-drill.html`

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data)

Demo: sample project walk-through

PREDICT FIRST

Which alert will the risk hook refuse, and which rule fires?

```
$ npm run demo -- --recorded --max-notional 800
```

YOU SHOULD SEE

- one decision at a time: order, oracle, rule
- the failure test refusing four bad orders

Fallback: Same recorded data. The decisions are the same every run.



Lab demo, runs from disk, recorded data.

`Try it: https://nus-solana-demos.vercel.app/0009/04-sample-walkthrough.html`

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data)

Try the demos yourself

DEMOS

`nus-solana-demos.vercel.app/0009/`

Open on your laptop. Run your own pitch against the timer.

No install. The same files are in the repo and open from disk.



Scan to open the demos

Source: demo-repo/lab (the four lab demos, deployed as static pages, recorded data)

Your mission

Rehearsal 1 and the fix block, 18:35 to 19:35.

1

Time

Run your 3 minutes on the timer

2

Listen

Peers note one unclear claim

3

Fix

The weakest claim, then rerun the core test

4

Backup

Play your recording with Wi-Fi off

Finish line: one claim that a listener can restate.

Build menu

Between rehearsals. Pick one, or fix your own weakest claim.

BEGINNER	Tighten the script to the budget
STANDARD	Add the missing evidence to the pack
STANDARD	Rehearse offline from the recording
ADVANCED	Add a test for the failure you demo

Three takeaways

1. Use the same evidence in the pitch, the repo and the running demo.
2. Say what works, what failed and what is untested.
3. A good answer points to a test or admits a limitation.

Branch out

Before Demo Day, in your own time.

BEGINNER

Time your pitch three times and trim the part that always runs long.

STANDARD

Fill your own demo pack in the rubric and fix the first missing check.

ADVANCED

Write one funded devnet transaction per decision and open the link in your pitch.

Share back

Before you leave, and by Demo Day.

TONIGHT

1

Freeze the commit and send the demo pack.

THU 18 FEB

2

Present at Demo Day, same evidence, same script.

ANY TIME

3

Send your repo link to the leads.

Get the code

REPO

`github.com/travisim/solana-agentic-trading-nus`

`folder: 0009-demo-rehearsal/demo`

```
1  $ npm install
```

```
2  $ npm run demo -- --recorded
```

Run both from the demo/code folder. Node 24. No keys, no AI credits. Recorded data is bundled.



Scan for this session's code

Let's connect



LinkedIn

linkedin.com/in/travisim



Telegram

t.me/travisim


SOLANA